

8 Mapping collections and entity associations

This chapter covers

- Mapping persistent collections
- Examining collections of basic and embeddable types
- Investigating simple many-to-one and one-to-many entity associations

The first thing many developers try to do when they begin using Hibernate or Spring Data JPA is to map a *parent/child relationship*. This is usually the first time they encounter collections. It's also the first time they have to think about the differences between entities and value types or get lost in the complexity of ORM.

Managing the associations between classes and the relationships between tables is at the heart of ORM. Most of the difficult problems involved in implementing an ORM solution relate to collections and entity association management. We'll start this chapter with some basic collection-mapping concepts and simple examples. After that, you'll be prepared for the first collection in an entity association—we'll come back to more complicated entity association mappings in the next chapter. To get the full picture, we recommend you read both this chapter and the next.

Major new features in JPA 2

Support was added for collections and maps of basic and embeddable types.

Support was added for persistent lists where the index of each element is stored in an additional database column.

One-to-many associations now have an orphan removal option.

8.1 Sets, bags, lists, and maps of value types

Java has a rich collection API, from which we can choose the interface and implementation that best fits the domain model design. We'll use the Java Collections framework for our implementation in this chapter, and we'll walk through the most common collection mappings, repeating the same `Image` and `Item` example with minor variations.

We'll start by looking at the database schema and creating and mapping a collection property in general. The database goes first, as it is generally designed first and our programs must work with it. Then we'll proceed to selecting a specific collection interface and mapping various collection types: a set, an identifier bag, a list, a map, and finally sorted and ordered collections.

8.1.1 The database schema

We'll extend `CaveatEmptor` to support attaching images to auction items. An item with an associated image is more interesting for the potential buyer. We'll ignore the Java code for now and consider only the database schema. The source code that follows can be found in the `mapping-collections` folder.

NOTE To execute the examples from the source code, you will first need to run the `Ch08.sql` script.

For the auction item and images example, assume that the image is stored somewhere on the filesystem and that we store just the filename in the database. When an image is deleted from the database, a separate process must delete the file from the disk.

We need an `IMAGE` table in the database to hold the images, or maybe just the filenames of images. This table will also have a foreign key column, say `ITEM_ID`, referencing the `ITEM` table. Look at the schema shown in figure 8.1.

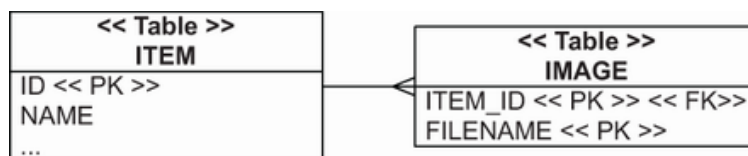


Figure 8.1 The `IMAGE` table holds image filenames, each referencing an `ITEM_ID`.

That's all there is to the schema—no collections or composition.

8.1.2 Creating and mapping a collection property

How would we map this `IMAGE` table with what we know so far? We'd probably map it as an `@Entity` class named `Image`. Later in this chapter we'll map a foreign key column with a `@ManyToOne` property to make the association between entities. We'd also need a composite primary key mapping for the entity class, as we'll first demonstrate in section 10.2.2. What we need to know for now is that a composite primary key is a combination of more than one column to uniquely identify a row in a table. Individual columns may not be unique, but their combination must be unique.

There are no mapped collections of images; they aren't necessary. When we need an item's images, we can write and execute a query in the JPA query language:

```
select img from Image img where img.item = :itemParameter
```

Persistent collections are *always* optional.

A collection we could create is `Item#images`, referencing all images for a particular item. We could create and map this collection property to do the following:

- Execute the SQL query `SELECT * from IMAGE where ITEM_ID = ?` automatically when we call `someItem.getImages()`. As long as the domain model instances are in a *managed* state (more on that later), we can read from the database on demand while navigating the associations between the classes. We don't have to manually write and execute a query to load data. On the other hand, when we start iterating the collection, the collection query is always "all images for this item," never "only images that match criteria XYZ."
- Avoid saving each `Image` with `entityManager.persist()` or `imageRepository.save()`. If we have a mapped collection, adding the `Image` to the collection with `someItem.getImages().add()` will make it persistent automatically when the `Item` is saved. This cascading persistence is convenient because we can save instances without calling the repository or the `EntityManager`.
- Have a dependent lifecycle of `Image`s. When an `Item` is deleted, Hibernate deletes all attached `Image`s with an extra SQL `DELETE`. We don't have to worry about the lifecycle of images and cleaning up orphans (assuming the database foreign key constraint doesn't `ON DELETE CASCADE`). The JPA provider handles the composition lifecycle.

It's important to realize that although these benefits sound great, the price we'll pay is additional mapping complexity. Many JPA beginners struggle with collection mappings, and frequently the answer to "Why are you doing this?" has been "I thought this collection was required."

If we analyze how we can treat the scenario with images for auction items, we'll find that we'll benefit from a collection mapping. The images have a dependent lifecycle; when an item is deleted, all the attached images should be deleted. When an item is stored, all attached images should be stored. And when an item is displayed, we'll often also display all images, so `someItem.getImages()` is convenient in UI code—this is rather an eager loading of the information. We don't have to call the persistence service again to get the images; they're just *there*.

Now we'll move on to choosing the collection interface and implementation that best fits the domain model design. Let's walk through the most common collection mappings, repeating the same `Image` and `Item` example with minor variations.

8.1.3 Selecting a collection interface

This is the idiom for a collection property in the Java domain model:

```
<<Interface>> images = new <<Implementation>>();  
// Getter and setter methods  
// . . .
```

Use an interface to declare the type of the property, not an implementation. Pick a matching implementation, and initialize the collection right away; doing so avoids uninitialized collections. We don't recommend initializing collections late in constructors or setter methods.

Using generics, here's a typical `Set` :

```
Set<Image> images = new HashSet<Image>();
```

Raw collections without generics

If we don't specify the type of collection elements with generics, or the key/value types of a map, we need to tell Hibernate the type (or types). For example, instead of a `Set<String>`, we can map a raw `Set` with `@ElementCollection(targetClass= String.class)`. This also applies to type parameters of a `Map`. Specify the key type of a `Map` with `@MapKeyClass`.

All the examples in this book use generic collections and maps, and so should you.

Out of the box, Hibernate supports the most important JDK collection interfaces and preserves the semantics of JDK collections, maps, and arrays in a persistent fashion. Each JDK interface has a matching implementation supported by Hibernate, and it's important that we use the right combination. Hibernate wraps the already initialized collection on the declaration of the field or sometimes replaces it if it's not the right one. It does that to enable, among other things, lazy loading and dirty checking of collection elements.

Without extending Hibernate, we can choose from the following collections:

- A `java.util.Set` property, initialized with a `java.util.HashSet`. The order of elements isn't preserved, and duplicate elements aren't allowed. All JPA providers support this type.
- A `java.util.SortedSet` property, initialized with a `java.util.TreeSet`. This collection supports a stable order of elements: sorting occurs in memory after Hibernate loads the data. This is a Hibernate-only extension; other JPA providers may ignore the “sorted” aspect of the set.
- A `java.util.List` property, initialized with a `java.util.ArrayList`. Hibernate preserves the position of each element with an additional index column in the database table. All JPA providers support this type.
- A `java.util.Collection` property, initialized with a `java.util.ArrayList`. This collection has *bag* semantics; duplicates are possible, but the order of elements isn't preserved. All JPA providers support this type.
- A `java.util.Map` property, initialized with a `java.util.HashMap`. The key and value pairs of a map can be preserved in the database. All JPA providers support this type.
- A `java.util.SortedMap` property, initialized with a `java.util.TreeMap`. It supports a stable order of elements: sorting occurs in memory after Hibernate loads the data. This is a Hibernate-only extension; other JPA providers may ignore the “sorted” aspect of the map.
- Hibernate supports persistent arrays, but JPA doesn't. They're rarely used, and we won't show them in this book. Hibernate can't wrap array properties, so many benefits of collections, such as on-demand lazy loading, won't work. Only use persistent arrays in your domain model if you're sure you won't need lazy loading. (You can load arrays on demand, but this requires interception with bytecode enhancement, as explained in section 12.1.3.)

If we want to map collection interfaces and implementations that are not directly supported by Hibernate, we need to tell Hibernate about the semantics of the custom collections. The extension point in Hibernate is the `PersistentCollection` interface in the `org.hibernate.collection.spi` package, where we usually extend one of the existing `PersistentSet`, `PersistentBag`, and `PersistentList` classes. Custom persistent collections aren't easy to write, and we don't recommend doing this if you aren't an experienced Hibernate user.

Transactional filesystems

If we only keep the filenames of images in the SQL database, we have to store the binary data of each picture—the files—somewhere. We could store the image data in the SQL database in `BLOB` columns (see “Binary and large value types” in section 6.3.1).

If we decide not to store the images in the database, but as regular files, we should be aware that the standard Java filesystem APIs, `java.io.File` and `java.nio.file.Files`, aren't transactional. Filesystem operations aren't enlisted in a Java Transaction API (JTA) system transaction; a transaction might successfully complete with Hibernate writing the filename into the SQL database, but the storing or deleting of the file in the filesystem might fail. We won't be able to roll back these operations as one atomic unit, and we won't get proper isolation of operations.

You could use a separate system transaction manager such as Bitronix. File operations are then enlisted, committed, and rolled back together with Hibernate's SQL operations in the same transaction.

Let's map a collection of image filenames for an `Item`.

8.1.4 Mapping a set

The simplest implementation of mapping a set is a `Set` of `String` image filenames. Add a collection property to the `Item` class, as demonstrated in the following listing.

Listing 8.1 Images mapped as a simple set of strings

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofstrings/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection                                ①
    @CollectionTable(
        name = "IMAGE",                                ②
        joinColumns = @JoinColumn(name = "ITEM_ID"))    ③
    @Column(name = "FILENAME")                          ④
    private Set<String> images = new HashSet<>();        ⑤
```

① Declare the `images` field as an `@ElementCollection`. We refer here to image paths on the system, but for brevity we'll use the names of the fields and columns, such as `image` or `images`.

② The collection table will be named `IMAGE`. Otherwise it would default to `ITEM_IMAGES`.

③ The join column between the `ITEM` and the `IMAGE` tables will be `ITEM_ID` (the default name, in fact).

① The name of the column that will contain the string information from the `images` collection will be `FILENAME`. Otherwise it would default to `IMAGES`.

② Initialize the `images` collection as a `HashSet`.

The `@ElementCollection` JPA annotation in the preceding listing is required for a collection of value-typed elements. Without the `@CollectionTable` and `@Column` annotations, Hibernate would use default schema names. Look at the schema in figure 8.2: the primary key columns are underlined.

ITEM		IMAGE	
<u>ID</u>	NAME	<u>ITEM_ID</u>	<u>FILENAME</u>
1	Foo	1	landscape.jpg
		1	foreground.jpg
		1	background.jpg
		1	portrait.jpg

Figure 8.2 Table structure and example data for a set of strings

The `IMAGE` table has a composite primary key of both the `ITEM_ID` and `FILENAME` columns. That means we can't have duplicate rows: each image file can only be attached once to one item. Also, the order of images isn't stored. This fits the domain model and `Set` collection. The image is stored somewhere on the filesystem, and we keep just the filename in the database.

To interact with the `Item` entities, we'll create the following Spring Data JPA repository.

Listing 8.2 The `ItemRepository` interface

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/repositories/setofstrings/ItemRepository.java

public interface ItemRepository extends JpaRepository<Item, Long> {

    @Query("select i from Item i inner join fetch i.images where i.id = :
    Item findItemWithImages(@Param("id") Long id);

    @Query(value = "SELECT FILENAME FROM IMAGE WHERE ITEM_ID = ?1",
            nativeQuery = true)
    Set<String> findImagesNative(Long id);
}
```

③ Declare a `findItemWithImages` method that will get the `Item` by `id`, including the `images` collection. To fetch this collection eagerly,

we'll use the inner join fetch capability of Jakarta Persistence Query Language (JPQL).

Ⓑ Declare the `findImagesNative` method, which is annotated as a native query and will get the set of strings representing the `images` of a given `id`.

We'll also create the following test.

Listing 8.3 The `MappingCollectionsSpringDataJPATest` class

```
Path: Ch08/mapping-collections/src/test/java/com/manning/javapersistenc
➡ /ch08//setofstrings/MappingCollectionsSpringDataJPATest.java

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class MappingCollectionsSpringDataJPATest {

    @Autowired
    private ItemRepository itemRepository;

    @Test
    void storeLoadEntities() {

        Item item = new Item("Foo");

        item.addImage("background.jpg");
        item.addImage("foreground.jpg");
        item.addImage("landscape.jpg");
        item.addImage("portrait.jpg");

        itemRepository.save(item);

        Item item2 = itemRepository.findItemWithImages(item.getId());

        List<Item> items2 = itemRepository.findAll();
        Set<String> images = itemRepository.findImagesNative(item.getId

        assertAll(
            () -> assertEquals(4, item2.getImages().size()),
            () -> assertEquals(1, items2.size()),
            () -> assertEquals(4, images.size())
        );

    }
}
```

Ⓐ Create an `Item`.

Ⓑ Add 4 image paths to it.

- Ⓒ Save it to the database.
- Ⓓ Access the repository to get the item together with the `images` collection. As we specified in the JPQL query with which the `findItemWithImages` method is annotated, the collection will also be fetched from the database.
- Ⓔ Get all `Items` from the database.
- Ⓕ Get the set of strings representing the images, using the `findImagesNative` method.
- Ⓖ Check the sizes of the different collections we have obtained.

It doesn't seem likely that we'd allow the user to attach the same image more than once to the same item, but let's suppose we did. What kind of mapping would be appropriate in that case?

8.1.5 Mapping an identifier bag

A *bag* is an unordered collection that allows duplicate elements, like the `java.util.Collection` interface. Curiously, the Java Collections framework doesn't include a bag implementation. We can initialize the property with an `ArrayList`, and Hibernate ignores the index of elements when storing and loading elements.

Listing 8.4 Bag of strings, allowing duplicate elements

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/bagofstrings/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @Column(name = "FILENAME")
    @GenericGenerator(name = "sequence_gen", strategy = "sequence")
    @org.hibernate.annotations.CollectionId(
        columns = @Column(name = "IMAGE_ID"),
        type = @org.hibernate.annotations.Type(type = "long"),
        generator = "sequence_gen")
    private Collection<String> images = new ArrayList<>();
```

- Ⓐ Declare a `@GenericGenerator` with the name `"sequence_gen"` and the `"sequence"` strategy to take care of the surrogate keys in the `IMAGE` table.

Ⓑ The `IMAGE` collection table needs a different primary key to allow duplicate `FILENAME` values for each `ITEM_ID`.

Ⓒ Introduce a surrogate primary key column named `IMAGE_ID`. You may retrieve all images at the same time or store them all at the same time, but a database table still needs a primary key.

Ⓓ Use a Hibernate-only annotation.

Ⓔ Configure how the primary key is generated.

Ⓕ There is no bag implementation in JDK. We initialize the collection as `ArrayList`.

Usually you'll want the system to generate a primary key value when you save an entity instance. If you need to refresh your memory about key generators, refer to section 5.2.4. The modified schema is shown in figure 8.3. The Spring Data JPA repository and the test will be just like in the previous example.

ITEM		IMAGE		
<u>ID</u>	NAME	<u>IMAGE_ID</u>	ITEM_ID	FILENAME
1	Foo	1	1	landscape.jpg
		2	1	foreground.jpg
		3	1	background.jpg
		4	1	portrait.jpg

Figure 8.3 Surrogate primary key column for a bag of strings

Here's an interesting question: if all you see is this schema, can you tell how the tables are mapped in Java? The `ITEM` and `IMAGE` tables look the same: each has a surrogate primary key column and some other normalized columns. Each table could be mapped with an `@Entity` class. However, we could decide to use a JPA feature and map a collection to `IMAGE`, even with a composition lifecycle. This is, effectively, a design decision that some predefined query and manipulation rules are all we need for this table instead of the more generic `@Entity` mapping. When you make such a decision, be sure you know the reasons why and the consequences.

The next mapping technique preserves the order of images within a list.

8.1.6 Mapping a list

If you haven't used ORM software before, a persistent list seems to be a very powerful concept; imagine how much work storing and loading a `java.util.List<String>` is with plain JDBC and SQL. If we add an element to the middle of the list, the list shifts all subsequent elements to

the right or rearranges pointers, depending on the list implementation. If we remove an element from the middle of the list, something else happens, and so on. If the ORM software can do all of this automatically for database records, a persistent list starts to look more appealing than it actually is.

As we noted in section 3.2.4, the first reaction is often to preserve the order of data elements as users enter them because you'll often have to show them later in the same order. But if another criterion can be used for sorting the data, like an entry timestamp, you should sort the data when querying rather than store the display order. What if the display order you need to use changes? The order in which data is displayed is usually not an integral part of the data but an orthogonal concern, so think twice before mapping a persistent `List`; Hibernate isn't as smart as you might think, as you'll see in the next example.

Let's change the `Item` entity and its collection property.

Listing 8.5 Persistent list, preserving the order of elements in the database

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/listofstrings/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @OrderColumn // Enables persistent order, Defaults to IMAGES_ORDER
    @Column(name = "FILENAME")
    private List<String> images = new ArrayList<>();
```

There is a new annotation in this example: `@OrderColumn`. This column stores an index in the persistent list, starting at zero. The column name will default to `IMAGES_ORDER`. Note that Hibernate stores the index to be contiguous in the database and expects it to be that way. If there are gaps, Hibernate will add `null` elements when loading and constructing the `List`. Look at the schema in figure 8.4.

ITEM		IMAGE		
ID	NAME	ITEM_ID	IMAGES_ORDER	FILENAME
1	Foo	1	0	landscape.jpg
2	Bar	1	1	foreground.jpg
		1	2	background.jpg
		1	3	background.jpg
		2	0	portrait.jpg
		2	1	foreground.jpg

Figure 8.4 The collection table preserves the position of each list element.

The primary key of the `IMAGE` table is a composite of `ITEM_ID` and `IMAGES_ORDER`. This allows duplicate `FILENAME` values, which is consistent with the semantics of a `List`. Remember, the image is stored somewhere on the filesystem, and we keep just the filename in the database. The Spring Data JPA repository and the test will be just like the previous example.

We said earlier that Hibernate isn't as smart as you might think. Consider making modifications to the list: say the list has three images, A, B, and C, in that order. What happens if you remove A from the list? Hibernate executes one SQL `DELETE` for that row. Then it executes two `UPDATE`s, for B and C, shifting their positions to the left to close the gap in the index. For each element to the right of the deleted element, Hibernate executes an `UPDATE`. If we wrote SQL for this by hand, we could do it with one `UPDATE`. The same is true for insertions in the middle of the list—Hibernate shifts all existing elements to the right one by one. At least Hibernate is smart enough to execute a single `DELETE` when we `clear()` the list.

Now suppose the images for an item have user-supplied names in addition to the filename. One way to model this in Java is with a map, using key/value pairs.

8.1.7 Mapping a map

To accommodate user-supplied names for the image files, we'll change the Java class to use a `Map` property.

Listing 8.6 Persistent map storing its key and value pairs

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofstrings/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
```

```

@CollectionTable(name = "IMAGE")
@MapKeyColumn(name = "FILENAME")
@Column(name = "IMAGENAME")
private Map<String, String> images = new HashMap<>();

```

A
B

A Each map entry is a key/value pair. Here the key is mapped with `@MapKeyColumn` to `FILENAME`.

B The value is the `IMAGENAME` column. This means the user can only use a file once because a `Map` doesn't allow duplicate keys.

As you can see from the schema in figure 8.5, the primary key of the collection table is a composite of `ITEM_ID` and `FILENAME`. The example uses a `String` as the key for the map, but Hibernate supports any basic type, such as `BigDecimal` or `Integer`. If the key is a Java `enum`, we must use `@MapKeyEnumerated`. With any temporal types such as `java.util.Date`, use `@MapKeyTemporal`.

ITEM		IMAGE		
<u>ID</u>	NAME	<u>ITEM_ID</u>	<u>FILENAME</u>	IMAGENAME
1	Foo	1	landscape.jpg	Landscape
2	Bar	1	foreground.jpg	Foreground
		1	background.jpg	Background
		1	portrait.jpg	Portrait
		2	landscape.jpg	Landscape
		2	foreground.jpg	Foreground

Figure 8.5 Tables for a map, using strings as indexes and elements

The map in the previous example was unordered. If the list of files is long, and we would like to quickly look for something at a glance, how can we always sort map entries by filename?

8.1.8 Sorted and ordered collections

We can *sort* a collection in memory by using a Java comparator. We can *order* a collection when it's loaded from the database by using an SQL query with an `ORDER BY` clause.

Let's make the map of images a sorted map. We need to change the Java property and the mapping.

Listing 8.7 Sorting map entries in memory using a comparator

```

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/sortedmapofstrings/Item.java

```

```

@Entity

```

```

public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @MapKeyColumn(name = "FILENAME")
    @Column(name = "IMAGENAME")
    @org.hibernate.annotations.SortComparator(ReverseStringComparator.c
    private SortedMap<String, String> images = new TreeMap<>();

```

Sorted collections are a Hibernate feature; hence the annotation `org.hibernate.annotations.SortComparator` with an implementation of `java.util.Comparator<String>` —the one shown here sorts strings in reverse order. The database schema doesn't change, which is also the case for all the following examples. Look at figures 8.1–8.5 in the previous sections if you need a reminder.

We'll add the following two lines to the test, which will check that the keys are now in reverse order:

```

() -> assertEquals("Portrait", item2.getImages().firstKey()),
() -> assertEquals("Background", item2.getImages().lastKey())

```

We'll map a `java.util.SortedSet` as demonstrated next. You can find it in the `sortedsetofstrings` example in the `mapping-collections` folder.

Listing 8.8 Sorting set elements in memory with `String#compareTo()`

```

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/sortedsetofstrings/Item.java

```

```

@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @Column(name = "FILENAME")
    @org.hibernate.annotations.SortNatural
    private SortedSet<String> images = new TreeSet<>();

```

Here natural sorting is used, falling back on the `String#compareTo()` method.

Unfortunately, we can't sort a bag; there is no `TreeBag`. The indexes of list elements predefine their order. Alternatively, instead of switching to `Sorted*` interfaces, we may want to retrieve the elements of a collection in the right order from the database, rather than sorting in memory.

Instead of a `java.util.SortedSet`, we'll use a `java.util.LinkedHashSet` in the following listing.

Listing 8.9 `LinkedHashSet` offers insertion order for iteration

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofstringsorderby/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @Column(name = "FILENAME")
    // @javax.persistence.OrderBy // One possible order: "FILENAME asc"
    @org.hibernate.annotations.OrderBy(clause = "FILENAME desc")
    private Set<String> images = new LinkedHashSet<>();
```

The `LinkedHashSet` class has a stable iteration order over its elements, and Hibernate will fill it in the right order when loading a collection. To do this, Hibernate applies an `ORDER BY` clause to the SQL statement that loads the collection. We must declare this SQL clause with the proprietary `@org.hibernate.annotations.OrderBy` annotation. We could call an SQL function, like `@OrderBy("substring(FILENAME, 0, 3) desc")`, which would sort by the first three letters of the filename, but be careful to check that the DBMS supports the SQL function you're calling. Furthermore, you can use the SQL:2003 syntax `ORDER BY . . . NULLS FIRST|LAST`, and Hibernate will automatically transform it into the dialect supported by your DBMS.

If the expression is just a column name with `ASC` or `DESC`, a `@javax.persistence.OrderBy` annotation works fine as well. If you need a more elaborate clause (such as the `substring()` example in the previous paragraph), an `@org.hibernate.annotations.OrderBy` annotation is required.

Hibernate's `@OrderBy` vs. JPA's `@OrderBy`

You can apply the annotation `@org.hibernate.annotations.OrderBy` to any collection; its parameter is a plain SQL fragment that Hibernate attaches to the SQL statement loading the collection.

Java Persistence has a similar annotation, `@javax.persistence.OrderBy`. Its only parameter is not SQL but `someProperty DESC|ASC`. A `String` or `Integer` element value has no properties, so when we apply JPA's `@OrderBy` annotation on a collection of a basic type, as in listing 8.9 with `Set<String>`, "the ordering will be by value of the basic objects,"

according to the specification. This means we can't change the ordering value (just the direction, `asc` or `desc`). We'll use the JPA annotation in section 8.2.2 when the element value class has persistent properties and isn't of basic/scalar type.

The next example from `bagofstringsorderby` demonstrates the same ordering at load time with a bag mapping. You can find it in the `mapping-collections` folder.

Listing 8.10 `ArrayList` provides a stable iteration order

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/bagofstringsorderby/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @Column(name = "FILENAME")
    @GenericGenerator(name = "sequence_gen", strategy = "sequence")
    @org.hibernate.annotations.CollectionId(
        columns = @Column(name = "IMAGE_ID"),
        type = @org.hibernate.annotations.Type(type = "long"),
        generator = "sequence_gen")
    @org.hibernate.annotations.OrderBy(clause = "FILENAME desc")
    private Collection<String> images = new ArrayList<>();
```

Finally, we can load ordered key/value pairs with a `LinkedHashMap`.

Listing 8.11 `LinkedHashMap` keeps key/value pairs in order

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofstringsorderby/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @MapKeyColumn(name = "FILENAME")
    @Column(name = "IMAGENAME")
    @org.hibernate.annotations.OrderBy(clause = "FILENAME desc")
    private Map<String, String> images = new LinkedHashMap<>();
```

Keep in mind that the elements of ordered collections are only in the desired order when they're loaded. As soon as we add or remove elements,

the iteration order of the collections might be different than “by filename”; they behave like regular linked sets, maps, or lists. We demonstrated the technical approach, but we need to be aware of its shortcomings and to conclude that these make it a somewhat unreliable solution.

In a real system, it’s likely that we’ll need to store more than just the image name and filename. We’ll probably need to create an `Image` class for extra information (such as title, width, and height). This is the perfect use case for a collection of components.

8.2 Collections of components

We mapped an embeddable component earlier: the `address` of a `User`. The example we’re working with in this chapter is different because an `Item` has many references to an `Image`, as shown in figure 8.6. The association in the UML diagram is a composition (the black diamond); hence, the referenced `Image`s are bound to the lifecycle of the owning `Item`.

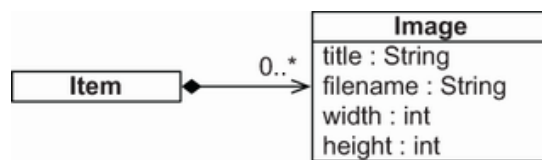


Figure 8.6 Collection of `Image` components in `Item`

The code in the following listing demonstrates the new `Image` embeddable class, capturing all the properties of an image that interest us.

Listing 8.12 Encapsulating all properties of an image

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddables/Image.java
```

```
@Embeddable
public class Image {
    @Column(nullable = false)
    private String filename;
    private int width;
    private int height;
    // . . .
}
```

First, note that all the properties are non-optional, `NOT NULL`. The size properties are non-nullable because their values are primitives. Second, we have to consider equality, and how the database and Java tier compare two images.

8.2.1 Equality of component instances

Let's say we want to keep several `Image` instances in a `HashSet`. We know that sets don't allow duplicate elements, but how do sets detect duplicates? The `HashSet` calls the `equals()` method on each `Image` we put in the `Set`. (It also calls the `hashCode()` method to get a hash, obviously.)

How many images are in the following collection?

```
someItem.addImage(new Image("background.jpg", 640, 480));
someItem.addImage(new Image("foreground.jpg", 800, 600));
someItem.addImage(new Image("landscape.jpg", 1024, 768));
someItem.addImage(new Image("landscape.jpg", 1024, 768));
assertEquals(3, someItem.getImages().size());
```

Did you expect four images instead of three? You're right: the regular Java equality check relies on identity. The

`java.lang.Object#equals()` method compares instances with `a==b`. Using this procedure, we'd have four instances of `Image` in the collection. Clearly, three is the "correct" answer for this use case.

For the `Image` class, we don't rely on Java identity—we override the `equals()` and `hashCode()` methods.

Listing 8.13 Implementing custom equality with `equals()` and `hashCode()`

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddables/Image.java

```
@Embeddable
public class Image {
    // . . .

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Image image = (Image) o;
        return width == image.width &&
            height == image.height &&
            filename.equals(image.filename) &&
            item.equals(image.item);
    }

    @Override
    public int hashCode() {
        return Objects.hash(filename, width, height, item);
    }
}
```

Ⓐ

Ⓑ

```
// . . .  
}
```

Ⓐ This custom equality check in `equals()` compares all values of one `Image` to the values of another `Image`. If all values are the same, the images must be the same.

Ⓑ The `hashCode()` method has to fulfill the contract requiring that if two instances are equal, they must have the same hash code.

Why didn't we override equality in section 6.2, when we mapped the `Address` of a `User`? Well, the truth is, we probably should have done that. Our only excuse is that we won't have any problems with the regular identity equality unless we put embeddable components into a `Set` or use them as keys in a `Map` that uses `equals()` and `hashCode()` for storing and comparison (this means it is not a `TreeMap`, which compares items with one another to sort and locate them). We should also redefine equality based on values, not identity. It's best if we override these methods on every `@Embeddable` class; all value types should be compared "by value."

Now consider the database primary key: Hibernate will generate a schema that includes all non-nullable columns of the `IMAGE` collection table in a composite primary key. The columns have to be non-nullable because we can't identify what we don't know. This reflects the equality implementation in the Java class. We'll look at the schema in the next section, with more details about the primary key.

NOTE There is a minor problem with Hibernate's schema generator: if we annotate an embeddable's property with `@NotNull` instead of `@Column(nullable=false)`, Hibernate won't generate a `NOT NULL` constraint for the collection table's column. A Bean Validation check of an instance works as expected, but the database schema is missing the integrity rule. Use `@Column(nullable=false)` if the embeddable class is mapped in a collection and the property should be part of the primary key.

The component class is now ready, and we can use it in collection mappings.

8.2.2 Set of components

We can map a `Set` of components as shown next. Keep in mind that a `Set` is a type of collection that allows only unique items.

Listing 8.14 A `Set` of embeddable components with an override

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddables/Item.java

```
@Entity
public class Item {
    // . . .

    @ElementCollection                                ①
    @CollectionTable(name = "IMAGE")                  ②
    @AttributeOverride(
        name = "filename",
        column = @Column(name = "FNAME", nullable = false)
    )
    private Set<Image> images = new HashSet<>();
}
```

① As before, the `@ElementCollection` annotation is required. Hibernate automatically knows that the target of the collection is an `@Embeddable` type from the declaration of a generic collection.

② The `@CollectionTable` annotation overrides the default name for the collection table, which would have been `ITEM_IMAGES`.

The `Image` mapping defines the columns of the collection table. Just as for a single embedded value, we can use `@AttributeOverride` to customize the mapping without modifying the target embeddable class.

Look at the database schema in figure 8.7. We’re mapping a set, so the primary key of the collection table is a composite of the foreign key column `ITEM_ID` and all “embedded” non-nullable columns: `FNAME`, `WIDTH`, and `HEIGHT`.

ITEM		IMAGE			
<u>ID</u>	NAME	<u>ITEM_ID</u>	<u>FNAME</u>	<u>WIDTH</u>	<u>HEIGHT</u>
1	Foo	1	landscape.jpg	640	480
2	Bar	1	foreground.jpg	800	600
		1	background.jpg	1024	768
		1	portrait.jpg	480	640
		2	landscape.jpg	640	480
		2	foreground.jpg	800	600

Figure 8.7 Example data tables for a collection of components

The `ITEM_ID` value wasn’t included in the overridden `equals()` and `hashCode()` methods of `Image`, as discussed in the previous section. Therefore, if we mix images of different items in one set, we’ll run into equality problems in the Java tier. In the database table, we obviously can distinguish the images of different items because the item’s identifier is included in primary key equality checks.

If we want to include the `Item` in the equality routine of the `Image`, to be symmetric with the database primary key, we'll need an `Image#item` property. This is a simple back-pointer provided by Hibernate when `Image` instances are loaded:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddables/Image.java
```

```
@Embeddable
public class Image {
    // . . .
    @org.hibernate.annotations.Parent
    private Item item;
    // . . .
}
```

We can now include the parent `Item` value in the `equals()` and `hashCode()` implementations.

In the next code snippet, we'll match the `FILENAME` field to the `FNAME` column using the `@AttributeOverride` annotation:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddables/Item.java
```

```
@AttributeOverride(
    name = "filename",
    column = @Column(name = "FNAME", nullable = false)
)
```

We'll similarly have to change the native query in the `ItemRepository` interface:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/repositories/setofembeddables/ItemRepository.java
```

```
@Query(value = "SELECT FNAME FROM IMAGE WHERE ITEM_ID = ?1",
    nativeQuery = true)
Set<String> findImagesNative(Long id);
```

If we need load-time ordering of elements and a stable iteration order with a `LinkedHashSet`, we can use the JPA `@OrderBy` annotation:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/setofembeddablesorderby/Item.java
```

```
@Entity
public class Item {
```

```
// . . .
@ElementCollection
@CollectionTable(name = "IMAGE")
@OrderBy("filename DESC, width DESC")
private Set<Image> images = new LinkedHashSet<>();
```

The arguments of the `@OrderBy` annotation are properties of the `Image` class, followed by either `ASC` for ascending or `DESC` for descending order. The default is ascending. This example sorts descending by image filename and then descending by the width of each image. Note that this is different from the proprietary `@org.hibernate.annotations.OrderBy` annotation, which takes a plain SQL clause, as discussed in section 8.1.8.

Declaring all properties of `Image` as `@NotNull` may not be something we want. If any of the properties are optional, we'll need a different primary key for the collection table.

8.2.3 Bag of components

We used the `@org.hibernate.annotations.CollectionId` annotation before adding a surrogate key column to the collection table. The collection type, however, was not a `Set` but a general `Collection`, a bag. This is consistent with our updated schema: if we have a surrogate primary key column, duplicate element values are allowed. Let's walk through this with the `bagofembeddables` example.

First, the `Image` class can now have nullable properties, as we'll have a surrogate key:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/bagofembeddables/Image.java
```

```
@Embeddable
public class Image {
    @Column(nullable = true)
    private String title;
    @Column(nullable = false)
    private String filename;
    private int width;
    private int height;
    // . . .
}
```

Remember to account for the optional `title` of the `Image` in the overridden `equals()` and `hashCode()` methods when comparing instances by value. For example, the comparison of the title fields will be done in the `equals` method like this:

```
Objects.equals(title, image.title)
```

Next, take a look at the mapping of the bag collection in `Item`. As before, in section 8.1.5, we declare an additional surrogate primary key column, `IMAGE_ID`, with the proprietary `@org.hibernate.annotations.CollectionId` annotation:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/bagofembeddables/Item.java
```

```
@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @GenericGenerator(name = "sequence_gen", strategy = "sequence")
    @org.hibernate.annotations.CollectionId(
        columns = @Column(name = "IMAGE_ID"),
        type = @org.hibernate.annotations.Type(type = "long"),
        generator = "sequence_gen")
    private Collection<Image> images = new ArrayList<>();
    // . . .
}
```

Figure 8.8 shows the database schema. The `title` of the `Image` with identifier 2 is `null`.

ITEM		IMAGE					
ID	NAME	IMAGE_ID	ITEM_ID	TITLE	FILENAME	WIDTH	HEIGHT
1	Foo	1	1	Landscape	landscape.jpg	640	480
2	Bar	2	1		foreground.jpg	800	600
		3	1	Background	background.jpg	1024	768
		4	1	Portrait	portrait.jpg	480	640
		5	2	Landscape	landscape.jpg	640	480
		6	2	Foreground	foreground.jpg	800	600

Figure 8.8 Collection of components table with a surrogate primary key column

Next, we'll analyze another way to change the primary key of the collection table with a `Map`.

8.2.4 Map of component values

A map keeps the information as pairs of keys and values. If the `Images` are stored in a `Map`, the filename can be the map key:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofstringembeddables/Item.java
```

```

@Entity
public class Item {
    // . . .

    @ElementCollection
    @CollectionTable(name = "IMAGE")
    @MapKeyColumn(name = "TITLE")
    private Map<String, Image> images = new HashMap<>();
    // . . .
}

```

Ⓐ The key column of the map is set to `TITLE`. Otherwise, it will default to `IMAGES_KEY`.

The test will set the `TITLE` column by executing instructions of this kind:

```

item.putImage("Background", new Image("background.jpg", 640, 480));

```

The primary key of the collection table, as shown in figure 8.9, is now the foreign key column `ITEM_ID` and the key column of the map, `TITLE`.

ITEM		IMAGE				
<u>ID</u>	NAME	<u>ITEM_ID</u>	<u>TITLE</u>	FILENAME	WIDTH	HEIGHT
1	Foo	1	Landscape	landscape.jpg	640	480
2	Bar	1	Foreground	foreground.jpg	800	600
		1	Background	background.jpg	1024	768
		1	Portrait	portrait.jpg	480	640
		2	Landscape	landscape.jpg	640	480
		2	Foreground	foreground.jpg	800	600

Figure 8.9 Database tables for a map of components

The embeddable `Image` class maps all other columns, which may be nullable:

```

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofstringembeddables/Image.java

```

```

@Embeddable
public class Image {
    @Column(nullable = true)
    private String filename;
    private int width;
    private int height;
    // . . .
}

```


Ⓐ The `filename` field can now be null; it is not part of the primary key.

Here the values in the map are instances of an embeddable component class and the keys of the map a basic string. Next we'll use embeddable types for both key and value.

8.2.5 Components as map keys

Our final example is mapping a `Map`, with both keys and values of embeddable type, as you can see in figure 8.10.

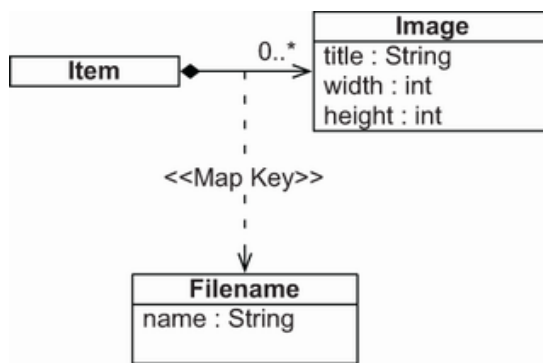


Figure 8.10 The `Item` has a `Map` keyed by `Filename`.

Instead of a string representation, we can represent a filename with a custom type.

Listing 8.15 Representing a filename with a custom type

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofembeddables/Filename.java

@Embeddable
public class Filename {
    @Column(nullable = false) Ⓐ
    private String name;
    // . . .
}
```

Ⓐ The `name` field must not be null, as it is part of the primary key. If we want to use this class for the keys of a map, the mapped database columns can't be nullable because they're all part of a composite primary key. We also have to override the `equals()` and `hashCode()` methods because the keys of a map are a set, and each `Filename` must be unique within a given key set.

We don't need any special annotations to map the collection:

```
Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/mapofembeddables/Item.java
```

```

@Entity
public class Item {
    @ElementCollection
    @CollectionTable(name = "IMAGE")
    private Map<Filename, Image> images = new HashMap<>();
    // . . .
}

```

In fact, we can't apply `@MapKeyColumn` and `@AttributeOverrides`; they have no effect when the map's key is an `@Embeddable` class.

The composite primary key of the `IMAGE` table includes the `ITEM_ID` and `NAME` columns, as you can see in figure 8.11. A composite embeddable class like `Image` isn't limited to simple properties of basic types. You've already seen how to nest other components, such as `City` in `Address`. We could extract and encapsulate the `width` and `height` properties of `Image` in a new `Dimensions` class.

ITEM		IMAGE				
<u>ID</u>	NAME	<u>ITEM_ID</u>	<u>NAME</u>	TITLE	WIDTH	HEIGHT
1	Foo	1	landscape.jpg	Landscape	640	480
2	Bar	1	foreground.jpg	Foreground	800	600
		1	background.jpg	Background	1024	768
		1	portrait.jpg	Portrait	480	640
		2	landscape.jpg	Landscape	640	480
		2	foreground.jpg	Foreground	800	600

Figure 8.11 Database tables for a `Map` of `Image`s keyed on `Filename`s

An embeddable class can also have its own collections.

8.2.6 Collection in an embeddable component

Suppose that for each `Address`, we want to store a list of contacts. This is a simple `Set<String>` in the embeddable class:

```

Path: Ch08/mapping-collections/src/main/java/com/manning/javapersistenc
➡ /ch08/embeddablesetofstrings/Address.java

```

```

@Embeddable
public class Address {
    @NotNull
    @Column(nullable = false)
    private String street;
    @NotNull
    @Column(nullable = false, length = 5)
    private String zipcode;
    @NotNull

```

```

@Column(nullable = false)
private String city;

@ElementCollection
@CollectionTable(
    name = "CONTACT",
    joinColumns = @JoinColumn(name = "USER_ID"))
@Column(name = "NAME", nullable = false)
private Set<String> contacts = new HashSet<>();
// . . .
}

```

Ⓐ The `@ElementCollection` is the only required annotation; the table and column names have default values. The table name would default to `USER_CONTACTS`.

Ⓑ The join column would be `USER_ID` by default.

Ⓒ The column name would default to `CONTACTS`.

Look at the schema in figure 8.12: the `USER_ID` column has a foreign key constraint referencing the owning entity's table, `USERS`. The primary key of the collection table is a composite of the `USER_ID` and `NAME` columns, preventing duplicate elements, so a `Set` is appropriate.

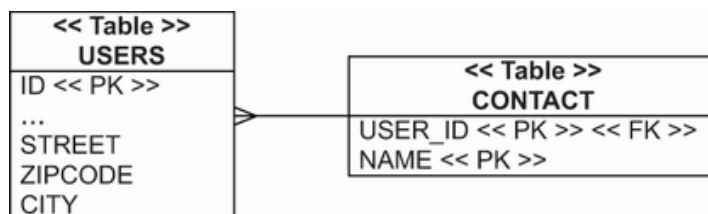


Figure 8.12 `USER_ID` has a foreign key constraint referencing `USERS`.

Instead of a `Set`, we could map a list, bag, or map of basic types. Hibernate also supports collections of embeddable types, so instead of a simple contact string, we could write an embeddable `Contact` class and have `Address` hold a collection of `Contacts`.

Although Hibernate gives a lot of flexibility with component mappings and fine-grained models, be aware that code is read more often than it's written. Think about the next developer who will have to maintain this in a few years.

Switching focus, let's turn our attention to entity associations: in particular, simple *many-to-one* and *one-to-many* associations.

8.3 Mapping entity associations

At the beginning of this chapter, we promised to talk about parent/child relationships. So far, we’ve looked at the mapping of an entity, `Item`. Let’s say this is the parent, and it has a collection of children: the collection of `Image` instances. The term *parent/child* implies some kind of lifecycle dependency, so a collection of strings or embeddable components is appropriate. The children are fully dependent on the parent; they will always be saved, updated, and removed with the parent, never alone.

We’ve already mapped a parent/child relationship! The parent was an entity, and the many children were of value type. When an `Item` is removed, its collection of `Image` instances will also be removed. (The actual images may be removed in a transactional way, meaning we’ll either delete the rows from the database together with the files from the disk, or nothing at all. This is, however, a separate problem that we won’t deal with here.)

We now want to map relationships of a different kind: associations between two entity classes. Their instances won’t have dependent lifecycles—one instance can be saved, updated, and removed without affecting another. Naturally, *sometimes* there will be dependencies, even between entity instances, but we’ll need more fine-grained control over how the relationship between the two classes affects instance state, unlike with completely dependent (embedded) types. Are we still talking about a parent/child relationship here? It turns out that the *parent/child* term is vague, and everyone has their own definition. We’ll try not to use that term from now on, and we’ll instead rely on more precise, or at least well-defined, vocabulary.

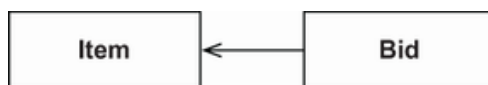


Figure 8.13 Relationship between `Item` and `Bid`

The relationship we’ll explore in the following sections will remain the same: a relationship between the `Item` and `Bid` entity classes, as shown in figure 8.13. The association from `Bid` to `Item` is a *many-to-one* association. Later we’ll make this association bidirectional, so the inverse association from `Item` to `Bid` will be *one-to-many*.

The *many-to-one* association is the simplest, so we’ll talk about it first. The other associations, *many-to-many* and *one-to-one*, are more complex, and we’ll discuss them in the next chapter.

Let’s start with the *many-to-one* association that we need to implement in the `CaveatEmptor` application, and let’s see what alternatives we have.

The source code that follows can be found in the mapping-associations folder.

8.3.1 The simplest possible association

We call the mapping of the `Bid#item` property a *unidirectional many-to-one association*. Before we analyze this mapping, look at the database schema in figure 8.14 and the code in listing 8.16.

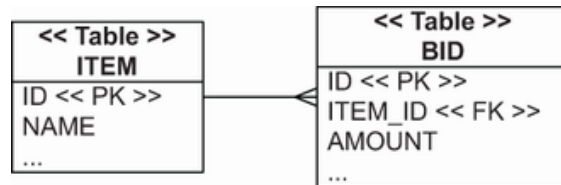


Figure 8.14 A many-to-one relationship in the SQL schema

Listing 8.16 `Bid` has a single reference to an `Item`

```
Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/bidirectional/Bid.java
```

```
@Entity
public class Bid {
    @ManyToOne(fetch = FetchType.LAZY)           ①
    @JoinColumn(name = "ITEM_ID", nullable = false)
    private Item item;
    // . . .
}
```

① The `@ManyToOne` annotation marks a property as an entity association, and it's required. Its `fetch` parameter defaults to `EAGER`, which means the associated `Item` is loaded whenever the `Bid` is loaded. We usually prefer lazy loading as a default strategy, and we'll talk more about it later in section 12.1.1.

A *many-to-one* entity association maps naturally to a foreign key column: `ITEM_ID` in the `BID` table. In JPA, this is called the *join column*. We don't need anything but the `@ManyToOne` annotation on the property. The default name for the join column is `ITEM_ID`: Hibernate automatically uses a combination of the target entity name and its identifier property, separated with an underscore.

We can override the foreign key column with the `@JoinColumn` annotation, but we used it here for a different reason: to make the foreign key column `NOT NULL` when Hibernate generates the SQL schema. A bid always has to have a reference to an item; it can't survive on its own. (Note that this already indicates some kind of lifecycle dependency we have to

keep in mind.) Alternatively, we could mark this association as non-optional with either `@ManyToOne(optional = false)` or, as usual, with Bean Validation's `@NotNull`.

That was easy. It's critically important to realize that we can write a complete and complex application without using anything else.

We don't need to map the other side of this relationship; we can ignore the *one-to-many* association from `Item` to `Bid`. There is only a foreign key column in the database schema, and we've already mapped it. We are serious about this: when you see a foreign key column and two entity classes involved, you should probably map it with `@ManyToOne` and nothing else.

We can now get the `Item` of each `Bid` by calling `someBid.getItem()`. The JPA provider will dereference the foreign key and load the `Item` for us, and it will also take care of managing the foreign key values. How can we get all of an item's bids? We can write a query and execute it with `EntityManager` or `JpaRepository` in whatever query language Hibernate supports. For example, in JPQL, we'd use `select b from Bid b where b.item = :itemParameter`. One of the reasons we use Hibernate or Spring Data JPA is, of course, that in most cases we don't want to write and execute that query ourselves.

8.3.2 Making it bidirectional

At the beginning of this chapter, in section 8.1.2, we had a list of reasons why a mapping of the `Item#images` collection was a good idea. Let's do the same for the `Item#bids` collection. This collection will implement the *one-to-many* association between `Item` and `Bid` entity classes. If we create and map this collection property, we'll get the following:

- Hibernate executes the SQL query `SELECT * from BID where ITEM_ID = ?` automatically when we call `someItem.getBids()` and start iterating through the collection elements.
- We can *cascade* state changes from an `Item` to all referenced `Bid`s in the collection. We can select what lifecycle events should be transitive; for example, we could declare that all referenced `Bid` instances should be saved when an `Item` is saved, so we don't have to call `EntityManager#persist()` or `ItemRepository#save()` repeatedly for all bids.

Well, that isn't a very long list. The primary benefit of one-to-many mappings is navigational access to data. It's one of the core promises of ORM, enabling us to access data by calling only methods of our Java domain model. The ORM engine is supposed to take care of loading the required data in a smart way while we work with a high-level interface of our own

design: `someItem.getBids().iterator().next().getAmount()`, and so on.

The fact that you can optionally cascade some state changes to related instances is a nice bonus. Consider, though, that some dependencies indicate value types at the Java level, and they do not indicate entities. Ask yourself if any table in the schema will have a `BID_ID` foreign key column. If not, map the `Bid` class as `@Embeddable`, not `@Entity`, using the same tables as before but with a different mapping with fixed rules for transitive state changes. If any other table has a foreign key reference on any `BID` row, we'll need a shared `Bid` entity; it can't be mapped embedded with an `Item`.

So should we map the `Item#bids` collection at all? We'll get navigational data access, but the price we'll pay is additional Java code and significantly more complexity. This is frequently a difficult decision; the choice of mapping the collection should rarely be taken. How often will we call `someItem.getBids()` in the application and then access or display all bids in a predefined order? If we only want to display a subset of the bids, or if we need to retrieve them in a different order every time, we'd need to write and execute queries manually anyway. The one-to-many mapping and its collection would only be maintenance baggage. In our experience, this is a frequent source of problems and bugs, especially for ORM beginners.

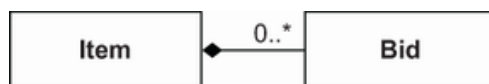


Figure 8.15 Bidirectional association between `Item` and `Bid`

In `CaveatEmptor`'s case, the answer is yes, we'll frequently call `someItem.getBids()` and then show a list to the user who wants to participate in an auction. Figure 8.15 shows the updated UML diagram with the bidirectional association that we need to implement.

The mapping of the collection and the one-to-many side is as follows.

Listing 8.17 `Item` has a collection of `Bid` references

```
Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/bidirectional/Item.java
```

```
@Entity
public class Item {
    // . . .
```

```
    @OneToMany(mappedBy = "item",                               ①
               fetch = FetchType.LAZY)                          ②
```

```

        private Set<Bid> bids = new HashSet<>();
        // . . .
    }

```

Ⓐ The `@OneToMany` annotation is required to make the association bidirectional. In this case, we also have to set the `mappedBy` parameter.

Ⓑ The argument is the name of the property on the “other side.” The fetching will be `LAZY` by default.

Look again at the other side—the many-to-one mapping in listing 8.16. The property name in the `Bid` class is `item`. The bid side is responsible for the foreign key column, `ITEM_ID`, which we mapped with `@ManyToOne`. Here, `mappedBy` tells Hibernate to “load this collection using the foreign key column already mapped by the given property”—in this case, `Bid#item`. The `mappedBy` parameter is always required when the one-to-many is bidirectional and we’ve already mapped the foreign key column. We’ll talk about that again in the next chapter.

The default for the `fetch` parameter of a collection mapping is always `FetchType.LAZY`, so we won’t need this option in the future. It’s a good default setting; the opposite would be the rarely needed `EAGER`. We don’t want all the bids eagerly loaded every time we load an `Item`. They should be loaded when accessed, on demand.

We can now create the following two Spring Data JPA repositories.

Listing 8.18 The `ItemRepository` interface

```

Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/repositories/onetomany/bidirectional/ItemRepository.java

public interface ItemRepository extends JpaRepository<Item, Long> {
}

```

Listing 8.19 The `BidRepository` interface

```

Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/repositories/onetomany/bidirectional/BidRepository.java

public interface BidRepository extends JpaRepository<Bid, Long> {
    Set<Bid> findByItem(Item item);
}

```

These are regular Spring Data JPA repositories, with the `BidRepository` adding a method to get the bids by `Item`.

The second reason for mapping the `Item#bids` collection is the ability to cascade state changes, so let's look at that.

8.3.3 Cascading state

If an entity state change can be cascaded across an association to another entity, we need fewer lines of code to manage relationships. But this may have a serious performance consequence.

The following code creates a new `Item` and a new `Bid` and then links them:

```
Item someItem = new Item("Some Item");
Bid someBid = new Bid(new BigDecimal("123.00"), someItem);
someItem.addBid(someBid);
```

We have to consider both sides of this relationship: the `Bid` constructor accepts an item that's used to populate `Bid#item`. To maintain the integrity of the instances in memory, we need to add the bid to `Item#bids`. Now the link is complete from the perspective of the Java code; all the references are set. If you aren't sure why you need this code, please refer to section 3.2.4.

Let's save the item and its bids in the database, first without and then with transitive persistence.

ENABLING TRANSITIVE PERSISTENCE

With the current mapping of `@ManyToOne` and `@OneToMany`, we need to write the following code to save a new `Item` and several `Bid` instances.

Listing 8.20 Managing independent entity instances separately

```
Path: Ch08/mapping-associations/src/test/java/com/manning/javapersistence
➡ /ch08/onetomany/bidirectional/MappingAssociationsSpringDataJPATest.java
```

```
Item item = new Item("Foo");
Bid bid = new Bid(BigDecimal.valueOf(100), item);
Bid bid2 = new Bid(BigDecimal.valueOf(200), item);

itemRepository.save(item);
item.addBid(bid);
item.addBid(bid2);
bidRepository.save(bid);
bidRepository.save(bid2);
```

When we create several bids, calling `EntityManager#persist()` or `BidRepository#save()` on each seems redundant. New instances are

transient and have to be made persistent. The relationship between `Bid` and `Item` doesn't influence their lifecycles. If `Bid` were to be a value type, the state of a `Bid` would automatically be the same as the owning `Item`. In this case, however, `Bid` has its own completely independent state.

We said earlier that fine-grained control is sometimes necessary to express the dependencies between associated entity classes; this is such a case. The mechanism for this in JPA is the `cascade` option. For example, to save all bids when the item is saved, map the collection as demonstrated next.

Listing 8.21 Cascading persistent state from `Item` to all bids

```
Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/cascadepersist/Item.java
```

```
@Entity
public class Item {
    // . . .

    @OneToMany(mappedBy = "item", cascade = CascadeType.PERSIST)
    private Set<Bid> bids = new HashSet<>();
    // . . .
}
```

Cascading options here intend to be transitive, so we use `CascadeType.PERSIST` for the `ItemRepository#save()` or `EntityManager#persist()` operation. We can now simplify the code that links items and bids and then saves them.

Listing 8.22 All referenced `bids` are automatically made persistent

```
Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/onetomany/cascadepersist/MappingAssociationsSpringDataJPATest.:
```

```
Item item = new Item("Foo");

Bid bid = new Bid(BigDecimal.valueOf(100), item);
Bid bid2 = new Bid(BigDecimal.valueOf(200), item);
item.addBid(bid);
item.addBid(bid2);

itemRepository.save(item);    ❶
```

❶ We save the bids automatically, but later. At commit time, Spring Data JPA using Hibernate examines the managed/persistent `Item` instance and looks into the `bids` collection. It then calls `save()` internally on each

of the referenced `Bid` instances, saving them as well. The value stored in the column `BID#ITEM_ID` is taken from each `Bid` by inspecting the `Bid#item` property. The foreign key column is mapped by `@ManyToOne` on that property.

The `@ManyToOne` annotation also has the `cascade` option. We won't use this often. For example, we can't really say, "when the bid is saved, also save the item." The item has to exist beforehand; otherwise the bid won't be valid in the database. Think about another possible `@ManyToOne` relationship: the `Item#seller` property. The `User` has to exist before they can sell an `Item`.

Transitive persistence is a simple concept, frequently useful with `@OneToMany` or `@ManyToMany` mappings. On the other hand, we have to apply transitive deletion carefully.

CASCADING DELETION

It seems reasonable that the deletion of an item implies the deletion of all the bids for the item, because they're not relevant alone. This is what the *composition* (the filled-out diamond) in the UML diagram means. With the current cascading options, we'll have to write the following code to delete an item:

```
Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/onetomany/cascadepersist/MappingAssociationsSpringDataJPATest.

Item retrievedItem = itemRepository.findById(item.getId()).get();

for (Bid someBid : bidRepository.findByItem(retrievedItem)) {
    bidRepository.delete(someBid);           Ⓐ
}

itemRepository.delete(retrievedItem);       Ⓑ
```

Ⓐ First we remove the bids.

Ⓑ Then we remove the `Item` owner.

JPA offers a cascading option to help with this. The persistence engine can remove an associated entity instance automatically.

Listing 8.23 Cascading removal from `Item` to all bids

```
Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/cascaderemove/Item.java

@Entity
```

```

public class Item {
    // . . .

    @OneToMany(mappedBy = "item",
                cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
    private Set<Bid> bids = new HashSet<>();
    // . . .
}

```

Just as before with `PERSIST`, the `delete()` operations on this association will be cascaded. If we call `ItemRepository#delete()` or `EntityManager#remove()` on an `Item`, Hibernate loads the `bids` collection elements and internally calls `remove()` on each instance:

```

Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/onetomany/cascaderemove/MappingAssociationsSpringDataJPATest.java

itemRepository.delete(item);

```

One line of code will be enough to also delete bids one by one.

This deletion process is inefficient, however: Hibernate or Spring Data JPA must always load the collection and delete each `Bid` individually. A single SQL statement would have the same effect on the database:

```
delete from BID where ITEM_ID = ?.
```

Nobody in the database has a foreign key reference on the `BID` table. Hibernate doesn't know this, however, and it can't search the whole database for any row that might have a linked `BID_ID` (that is, a `BID_ID` that is actually an `Item` foreign key).

If `Item#bids` was instead a collection of embeddable components, `someItem.getBids().clear()` would execute a single SQL `DELETE`. With a collection of value types, Hibernate assumes that nobody can possibly hold a reference to the bids, and removing only the reference from the collection makes it orphan removable data.

ENABLING ORPHAN REMOVAL

JPA offers a flag that enables the same behavior for `@OneToMany` (and only `@OneToMany`) entity associations.

Listing 8.24 Enabling orphan removal on a `@OneToMany` collection

```

Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/orphanremoval/Item.java

@Entity

```

```

public class Item {
    // . . .

    @OneToMany(mappedBy = "item",
                cascade = CascadeType.PERSIST, orphanRemoval = true)
    private Set<Bid> bids = new HashSet<>();
    // . . .
}

```

The `orphanRemoval=true` argument tells Hibernate that we want to permanently remove a `Bid` when it's removed from the collection.

We'll change the `ItemRepository` interface as in the following listing.

Listing 8.25 The modified `ItemRepository` interface

```

Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/repositories/onetomany/orphanremoval/ItemRepository.java

public interface ItemRepository extends JpaRepository<Item, Long> {

    @Query("select i from Item i inner join fetch i.bids where i.id = :id")
    Item findItemWithBids(@Param("id") Long id);

}

```

Ⓐ The new `findItemWithBids` method will get the `Item` by `id`, including the `bids` collection. To fetch this collection, we'll use the inner join fetch capability of JPQL.

Here is an example of deleting a single `Bid`:

```

Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/onetomany/orphanremoval/MappingAssociationsSpringDataJPATest.java

Item item1 = itemRepository.findItemWithBids(item.getId());
Bid firstBid = item1.getBids().iterator().next();
item1.removeBid(firstBid);

itemRepository.save(item1);

```

Hibernate or Spring Data JPA using Hibernate will monitor the collection, and on transaction commit will notice that we removed an element from the collection. Hibernate now considers the `Bid` to be orphaned. We have guaranteed that nobody else had a reference to it; the only reference was the one we just removed from the collection. Thus, Hibernate or Spring Data JPA using Hibernate will automatically execute an SQL `DELETE` to remove the `Bid` instance in the database.

We still won't get the `clear()` one-shot `DELETE`, as with a collection of components. Hibernate respects the regular entity-state transitions, and the bids are all loaded and removed individually.

Orphan removal is a questionable process. It's fine in this example, where there is no other table in the database with a foreign key reference on `BID`. There are no consequences to deleting a row from the `BID` table; the only in-memory references to bids are in `Item#bids`.

As long as all of this is true, there is no problem with enabling orphan removal. It's a convenient option when the presentation layer can remove an element from a collection to delete something. We only need to work with domain model instances, and we don't need to call a service to perform this operation.

But consider what happens when we create a `User#bids` collection mapping—another `@OneToMany`—as shown in figure 8.16. This is a good time to test your knowledge of Hibernate: what will the tables and schema look like after this change? (Answer: the `BID` table has a `BIDDER_ID` foreign key column, referencing `USERS`.)



Figure 8.16 Bidirectional associations between `Item`, `Bid`, and `User`

The test shown in the following listing won't pass.

Listing 8.26 No clean up in-memory references after database removal

```
Path: Ch08/mapping-associations/src/test/java/com/manning/javapersisten
➡ /ch08/onetomany/orphanremoval/MappingAssociationsSpringDataJPATest.ja

User user = userRepository.findUserWithBids(john.getId());
assertAll(
    () -> assertEquals(1, items.size()),
    () -> assertEquals(2, bids.size()),
    () -> assertEquals(2, user.getBids().size())
);
Item item1 = itemRepository.findItemWithBids(item.getId());
Bid firstBid = item1.getBids().iterator().next();
item1.removeBid(firstBid);
itemRepository.save(item1);
//FAILURE
//assertEquals(1, user.getBids().size());
assertEquals(2, user.getBids().size());
List<Item> items2 = itemRepository.findAll();
List<Bid> bids2 = bidRepository.findAll();
```

```

assertAll(
    () -> assertEquals(1, items2.size()),
    () -> assertEquals(1, bids2.size()),
    () -> assertEquals(2, user.getBids().size())
//FAILURE
//() -> assertEquals(1, user.getBids().size())
);

```

Hibernate or Spring Data JPA thinks the removed `Bid` is orphaned and deletable; it will be deleted automatically in the database, but we still hold a reference to it in the other collection, `User#bids`. The database state is fine when this transaction commits; the deleted row of the `BID` table contained both foreign keys, `ITEM_ID` and `BIDDER_ID`. But we now have an inconsistency in memory, because saying, “Remove the entity instance when the reference is removed from the collection” naturally conflicts with shared references.

Instead of orphan removal, or even `CascadeType.REMOVE`, always consider a simpler mapping. Here, `Item#bids` would be fine as a collection of components, mapped with `@ElementCollection`. The `Bid` would be `@Embeddable` and have a `@ManyToOne` `bidder` property, referencing a `User`. (Embeddable components can own unidirectional associations to entities.)

This would provide the lifecycle we’re looking for: a full dependency on the owning entity. We’ll have to avoid shared references; the UML diagram in figure 8.16 makes the association from `Bid` to `User` unidirectional. Drop the `User#bids` collection—we don’t need this `@OneToMany`. If we need all the bids made by a user, we can write a query: `select b from Bid b where b.bidder = :userParameter`. (In the next chapter, we’ll complete this mapping with a `@ManyToOne` in an embeddable component.)

ENABLING ON DELETE CASCADE ON THE FOREIGN KEY

All the removal operations we’ve shown so far are inefficient. Bids have to be loaded into memory, and many SQL `DELETE`s are necessary. SQL databases support a more efficient foreign key feature: the `ON DELETE` option. In DDL, it looks like this: `foreign key (ITEM_ID) references ITEM on delete cascade` for the `BID` table.

This option tells the database to maintain the referential integrity of composites transparently for all applications accessing the database. Whenever we delete a row in the `ITEM` table, the database will automatically delete any row in the `BID` table with the same `ITEM_ID` key value. We only need one `DELETE` statement to remove all dependent data recursively, and nothing has to be loaded into application (server) memory.

You should check whether your schema already has this option enabled on foreign keys. If you want this option added to the Hibernate-generated schema, use the Hibernate `@OnDelete` annotation.

You should also check whether this option works with your DBMS and if Hibernate or Spring Data JPA using Hibernate generates a foreign key with the `ON DELETE CASCADE` option. This does not work with MySQL, so we chose to demonstrate this particular example on the H2 database. You will find it this way in the source code (Maven dependency in `pom.xml` and Spring Data JPA configuration).

Listing 8.27 Generating foreign key `ON DELETE CASCADE` in the schema

```
Path: Ch08/mapping-associations/src/main/java/com/manning/javapersisten
➡ /ch08/onetomany/ondeletescascade/Item.java
```

```
@Entity
public class Item {
    // . . .

    @OneToMany(mappedBy = "item", cascade = CascadeType.PERSIST)
    @org.hibernate.annotations.OnDelete(
        action = org.hibernate.annotations.OnDeleteAction.CASCADE
    )
    private Set<Bid> bids = new HashSet<>();    ❸
    // . . .
}
```

❸ One of the Hibernate’s quirks is visible here: the `@OnDelete` annotation affects only the schema generation by Hibernate. Settings that affect schema generation are usually on the “other” `mappedBy` side, where the foreign key/join column is mapped. The `@OnDelete` annotation is usually next to the `@ManyToOne` in `Bid`. When the association is mapped bidirectionally, however, Hibernate will only recognize it on the `@OneToMany` side.

Enabling foreign key cascade deletion in the database doesn’t influence Hibernate’s runtime behavior. We can still run into the same problem as shown in listing 8.26. Data in memory may no longer accurately reflect the state in the database. If all related rows in the `BID` table are automatically removed when a row in the `ITEM` table is deleted, the application code is responsible for cleaning up references and catching up with the database state. If we aren’t careful, we may even end up saving something that we or someone else previously deleted.

The `Bid` instances don’t go through the regular lifecycle, and callbacks such as `@PreRemove` have no effect. Additionally, Hibernate doesn’t automatically clear the optional second-level global cache, which potentially

contains stale data. Fundamentally, the kinds of problems encountered with database-level foreign key cascading are the same as when another application besides ours is accessing the same database, or any other database trigger makes changes. Hibernate can be a very effective utility in such a scenario, but there are other moving parts to consider.

If you work on a new schema, the easiest approach is to not enable database-level cascading and to map a composition relationship in your domain model as embedded/ embeddable, not as an entity association.

Hibernate or Spring Data JPA using Hibernate can then execute efficient SQL `DELETE` operations to remove the entire composite. We made this recommendation in the previous section: if you can avoid shared references, map the `Bid` as an `@ElementCollection` in `Item`, not as a standalone entity with `@ManyToOne` and `@OneToMany` associations. Alternatively, of course, you might not map any collections at all and use only the simplest mapping: a foreign key column with `@ManyToOne`, unidirectional between `@Entity` classes.

Summary

- Using simple collection mappings, such as a `Set<String>`, you can work through a rich set of interfaces and implementations.
- You can use sorted collections as well as Hibernate's options for letting the database return the collection elements in the desired order.
- You can use complex collections of user-defined embeddable types and sets, bags, and maps of components.
- You can use components as both keys and values in maps.
- You can use a collection in an embeddable component.
- Mapping the first foreign key column to an entity many-to-one association makes it bidirectional as a one-to-many. You can implement cascading options.